2	`useEffect`—Essential for handling side effects (API calls, timers, DOM updates). Learn how the dependency array works (empty, specific values, or omitted).
3	`useContext`—Important for accessing shared state without prop drilling. Frequently discussed in state management interviews.
4	`useMemo`—Optimizes performance by memoizing expensive computations. Commonly asked in performance-related interview questions.
5	`useCallback`—Prevents unnecessary re-renders by memoizing functions. Often paired with `useMemo` in interview scenarios.
6	`useRef`—Useful for accessing DOM elements or persisting values across renders without triggering re-renders.
7	`useReducer`—Manages complex state logic, similar to Redux. Often asked when dealing with scalable applications.
8	`useFormStatus`—Imported from `react-dom` instead of `react`, it is used in form handling to track the status of form actions. Important for Server Components and progressive form submission.

While these high-priority hooks form the foundation of React development, medium-priority hooks in table A.8 cover specialized use cases and advanced scenarios. These are valuable to understand but are less frequently emphasized in interviews.

Table A.8 Medium-priority React Hooks for frontend interview preparation

Priority	Hooks
9	`useLayoutEffect`—Runs synchronously after DOM mutations. Useful for specific scenarios but less common than `useEffect`.
10	`useActionState`—Updates state based on the result of a form action. Simplifies state management, enhances user experience with loading indicators, and works seamlessly with React Server